PocketMind — Complete Micro-LLM Build Specification

> 🎯 **Mission:** Build a compact, memory-augmented language model from first principles that can understand developer requests, retrieve knowledge from local projects, and generate safe structured actions. The project prioritizes learning, measurable reliability, and practical utility over reproducing a conventional large chatbot.

Document status: Implementation-ready specification

Primary language: Python 3.11+

Core framework: PyTorch 2.x

Target model: 40–50M parameters

License recommendation: Apache-2.0 for code; track dataset licenses separately

---

1. Product definition

1.1 What PocketMind is

PocketMind is a local developer intelligence system built around a small language model. It combines:

- A decoder-style neural language model implemented from first principles.
- Alternating local-attention and gated recurrent-memory layers.
- An external document memory backed by SQLite.
- Lexical retrieval initially, followed by a learned retrieval head.
- Grammar-constrained JSON generation for reliable actions.
- A correction store that turns user feedback into future training data.

The system should answer questions about indexed projects, summarize retrieved code or documentation, and translate natural-language requests into validated read-only operations.

1.2 Example behavior

Request

```
Find unfinished TODOs in the project and group them by module.
```

Generated action

```
{
  "version": "1",
  "action": "search_text",
  "arguments": {
    "pattern": "TODO|FIXME",
    "path": ".",
    "extensions": [".py", ".ts", ".tsx", ".md"]
  }
}
```

The deterministic runtime validates and executes the action, retrieves matching passages, and gives the results back to the model for synthesis.

1.3 Success criteria

- The complete model and training loop are implemented without using a prebuilt Transformer model.
- A tiny configuration can intentionally overfit a small sample.
- Validation loss decreases consistently on the selected corpus.
- At least 98% of constrained action outputs are syntactically valid JSON.
- At least 90% of actions on the held-out action test set use the correct action type.
- Retrieved-context answers outperform closed-book answers on the project QA benchmark.
- The runtime never executes an action that fails schema or policy validation.
- Training can resume deterministically from a checkpoint.

1.4 Non-goals

- Competing with general-purpose frontier models.
- Memorizing broad world knowledge.
- Unrestricted shell access.
- Training on unverified scraped data.
- Implementing CUDA kernels, autograd, or a tensor library in the first release.
- Hiding retrieval failures behind confident generated answers.

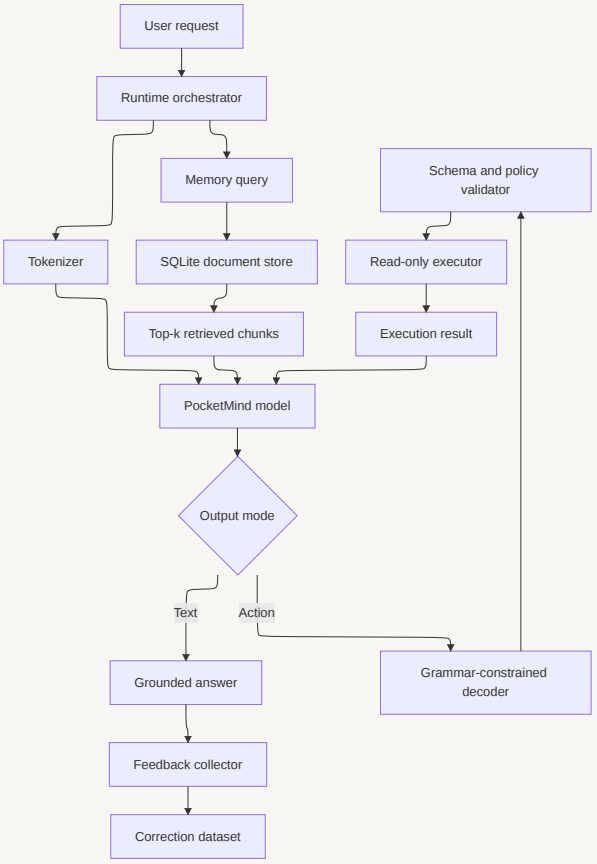
---

2. Guiding principles

1. **Small model, strong system.** Use model weights for language patterns and decisions; use external memory for knowledge.
2. **Learn before optimizing.** Implement clear reference versions before replacing them with optimized primitives.
3. **Constrain important outputs.** Reliability should not depend only on the model learning perfect JSON.
4. **Measure every claim.** Every architectural improvement needs an ablation or benchmark.
5. **Separate development and training profiles.** The same repository must support a tiny correctness profile and a larger training profile.
6. **Default to read-only.** Any state-changing capability belongs in a later, explicitly approved security phase.
7. **Data quality over data volume.** Deduplication, provenance, licensing, and clean validation splits are mandatory.

---

3. System architecture



3.1 Neural architecture

```
Token IDs
  → token embedding + positional representation
  → 12 alternating sequence blocks
  → final RMSNorm
  → language-model projection
  → next-token logits

Selected hidden states
  → retrieval projection
  → normalized query embedding
```

Recommended full configuration:

```yaml
vocab_size: 8000
context_length: 512
d_model: 512
n_layers: 12
n_heads: 8
```

```yaml
head_dim: 64
ffn_dim: 1536
dropout: 0.10
attention_window: 128
recurrent_state_dim: 512
tie_embeddings: true
norm: rmsnorm
activation: silu
position_encoding: rope
```

Recommended layer schedule:

Layer	Type	Purpose
1–2	Local causal attention	Short-range syntax and token relationships
3	Gated recurrent memory	Carry compressed sequence state
4–5	Local causal attention	Compose local patterns
6	Gated recurrent memory	Refresh long-range state
7	Local causal attention	Contextual composition
8	Retrieval cross-attention	Fuse retrieved memory tokens
9	Local causal attention	Integrate retrieved evidence
10	Gated recurrent memory	Maintain final sequence state
11–12	Local causal attention	Prepare prediction representation

🧪 Treat this schedule as a hypothesis. Build an ordinary local-attention baseline first, then compare it against the hybrid model using identical data, token budget, optimizer, and parameter count.

---

4. Repository structure

```
pocketmind/
├── README.md
├── LICENSE
├── pyproject.toml
├── uv.lock
├── Makefile
├── .env.example
```

```
├── .gitignore
├── configs/
│   ├── debug.yaml
│   ├── small.yaml
│   ├── full.yaml
│   ├── actions.yaml
│   └── retrieval.yaml
├── data/
│   ├── raw/
│   ├── interim/
│   ├── processed/
│   │   ├── train.bin
│   │   ├── validation.bin
│   │   ├── test.bin
│   │   └── documents.sqlite3
│   ├── manifests/
│   └── tokenizer/
├── src/pocketmind/
│   ├── __init__.py
│   ├── config.py
│   ├── tokenizer/
│   │   ├── byte_tokenizer.py
│   │   ├── bpe.py
│   │   ├── train_bpe.py
│   │   └── serialization.py
│   ├── data/
│   │   ├── manifest.py
│   │   ├── cleaning.py
│   │   ├── deduplication.py
│   │   ├── packing.py
│   │   ├── memmap_dataset.py
│   │   └── action_generator.py
│   ├── model/
│   │   ├── embeddings.py
│   │   ├── rmsnorm.py
│   │   ├── rope.py
│   │   ├── attention_reference.py
│   │   ├── attention_optimized.py
│   │   ├── recurrent_memory.py
│   │   ├── cross_attention.py
│   │   ├── feed_forward.py
│   │   ├── block.py
│   │   ├── pocketmind.py
│   │   └── generation.py
│   ├── training/
│   │   ├── trainer.py
│   │   ├── schedule.py
│   │   ├── checkpoint.py
│   │   ├── metrics.py
│   │   └── distributed.py
│   ├── retrieval/
│   │   ├── chunker.py
│   │   ├── store.py
│   │   ├── tfidf.py
│   │   ├── learned_retriever.py
│   │   └── reranker.py
```

```
│   ├── actions/
│   │   ├── schema.py
│   │   ├── grammar.py
│   │   ├── constrained_decode.py
│   │   ├── policy.py
│   │   └── executor.py
│   ├── runtime/
│   │   ├── prompts.py
│   │   ├── orchestrator.py
│   │   ├── feedback.py
│   │   └── cli.py
│   └── evaluation/
│       ├── language.py
│       ├── actions.py
│       ├── retrieval.py
│       ├── grounding.py
│       └── benchmark.py
├── scripts/
│   ├── prepare_corpus.py
│   ├── train_tokenizer.py
│   ├── tokenize_corpus.py
│   ├── train.py
│   ├── evaluate.py
│   ├── index_workspace.py
│   └── chat.py
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── regression/
│   └── fixtures/
├── benchmarks/
│   ├── action_cases.jsonl
│   ├── retrieval_cases.jsonl
│   └── grounding_cases.jsonl
├── checkpoints/
└── runs/
```

---

5. Environment and reproducibility

5.1 Dependencies

```
[project]
requires-python = ">=3.11"
dependencies = [
  "torch>=2.3",
  "numpy>=1.26",
  "pydantic>=2.7",
  "pyyaml>=6.0",
```

```
  "rich>=13.7",
  "typer>=0.12",
  "tqdm>=4.66",
  "scikit-learn>=1.5",
  "pytest>=8.2",
  "safetensors>=0.4"
]
```

Use a lockfile and record these values with every run:

- Git commit hash.
- Configuration file and resolved configuration.
- Random seeds.
- Python, PyTorch, CUDA, and driver versions.
- Device name and available memory.
- Dataset manifest hash.
- Tokenizer hash.
- Checkpoint parent identifier.

5.2 Execution profiles

Debug profile

```yaml
vocab_size: 256
context_length: 128
d_model: 128
n_layers: 4
n_heads: 4
ffn_dim: 384
micro_batch_size: 2
gradient_accumulation: 4
max_steps: 500
```

Small profile

```yaml
vocab_size: 8000
context_length: 256
```

```yaml
d_model: 256
n_layers: 8
n_heads: 4
ffn_dim: 768
micro_batch_size: 4
gradient_accumulation: 8
max_steps: 20000
```

Full profile

```yaml
vocab_size: 8000
context_length: 512
d_model: 512
n_layers: 12
n_heads: 8
ffn_dim: 1536
micro_batch_size: 4
gradient_accumulation: 16
max_steps: 50000
precision: bf16
activation_checkpointing: true
```

The trainer must automatically fall back from BF16 to FP16 when BF16 is unavailable. It must print the resolved memory-relevant settings before allocating the model.

---

6. Tokenization

6.1 Phase A: byte tokenizer

```python
class ByteTokenizer:
    vocab_size = 256

    def encode(self, text: str) -> list[int]:
        return list(text.encode("utf-8"))
```

```
    def decode(self, ids: list[int]) -> str:
        return bytes(ids).decode("utf-8", errors="replace")
```

Use this tokenizer for the first correctness milestones because it is transparent and has no unknown token.

6.2 Phase B: byte-backed BPE

Implement BPE rather than importing a tokenizer trainer:

1. Represent input as UTF-8 byte symbols.
2. Count adjacent symbol pairs.
3. Select the most frequent valid pair.
4. Merge every non-overlapping occurrence.
5. Add the merged symbol to the vocabulary.
6. Repeat until 8,000 symbols exist or the minimum pair frequency is reached.
7. Save vocabulary, merge order, special token IDs, corpus hash, and version.

Required special tokens:

```
<PAD> <BOS> <EOS> <UNK> <USER> <ASSISTANT>
<MEMORY> </MEMORY> <ACTION> </ACTION> <RESULT> </RESULT>
```

`<UNK>` should exist for format completeness but should never be required because the base alphabet contains every byte.

6.3 Tokenizer tests

- [ ] Every valid UTF-8 sample round-trips.
- [ ] Empty input round-trips.
- [ ] Code, JSON, paths, Unicode, and malformed-byte fallbacks are covered.
- [ ] Encoding is deterministic after serialization.
- [ ] No merge crosses a protected special-token boundary.
- [ ] BPE produces fewer tokens than raw bytes on the held-out corpus.

---

7. Dataset engineering

7.1 Corpus composition

Recommended initial mixture:

Category	Share	Purpose
Permissively licensed code	40%	Programming syntax and structure
Technical documentation	25%	Explanatory language
General clean text	15%	Language fluency
Structured JSON examples	10%	Schema-following behavior
Synthetic request/action pairs	10%	Practical tool selection

Start with 10–50M tokens for pipeline validation. Scale only after the model, checkpointing, and evaluation suite are trustworthy. A later complete run can target approximately 100–300M tokens.

7.2 Data manifest

Every source must have a manifest entry:

```
{
  "source_id": "stable-id",
  "origin": "source location",
  "license": "SPDX identifier or review-required",
  "sha256": "content hash",
  "collected_at": "ISO-8601 timestamp",
  "split": "train",
  "category": "code",
  "included": true,
  "exclusion_reason": null
}
```

7.3 Cleaning pipeline

1. Normalize line endings and Unicode.
2. Remove binary and generated files.
3. Remove secrets using deterministic patterns and entropy checks.
4. Remove documents with excessive replacement characters.
5. Deduplicate exact documents by SHA-256.

6. Deduplicate near-identical documents using MinHash or normalized shingles.
7. Add source-boundary tokens.
8. Split by document or repository, never by arbitrary token windows.
9. Tokenize and pack sequences.
10. Write immutable memory-mapped token files and a manifest hash.

> 🔒 Never train on API keys, credentials, private keys, environment files, or unreviewed personal data. Retrieval indexes must apply the same exclusions.

7.4 Binary format

Use `uint16` token IDs for an 8,000-token vocabulary and `numpy.memmap` for loading. Store separate train, validation, and test files. The dataset returns shifted windows:

```
x = tokens[start : start + context_length]
y = tokens[start + 1 : start + context_length + 1]
```

Assert that `x[1:]` equals `y[:-1]` in dataset tests.

7.5 Synthetic action generation

Generate examples from typed templates and independently validate every generated target against the schema. Vary:

- Verbs and paraphrases.
- Paths, extensions, patterns, and limits.
- Positive and negative requests.
- Ambiguous requests that require clarification.
- Requests that policy must deny.
- Multi-step requests that must be decomposed.

Keep template families separated across train and test splits so evaluation measures generalization rather than memorization.

---

8. Model components

8.1 RMSNorm

For hidden vector `x`:

$$y = x \cdot \frac{1}{\sqrt{mean(x^2) + \epsilon}} \cdot g$$

Test numerical agreement against a simple float32 reference.

8.2 Rotary positional embeddings

Apply RoPE to query and key pairs before computing attention. Cache sine and cosine values by sequence length, device, and dtype. Test that cache extension preserves existing positions.

8.3 Reference causal attention

```
scores = q @ k.transpose(-2, -1)
scores = scores / math.sqrt(head_dim)
scores = scores.masked_fill(causal_mask == 0, float("-in
f"))
weights = torch.softmax(scores, dim=-1)
weights = dropout(weights)
output = weights @ v
```

The local mask permits each position to attend only to itself and the previous `attention_window - 1` positions.

Critical invariant: changing token `t+1` must not alter outputs at positions `0..t`.

8.4 Optimized attention

After the reference implementation passes tests, add an optimized backend using scaled dot-product attention. Both implementations must share projections and produce close outputs with dropout disabled.

8.5 Gated recurrent-memory layer

Reference formulation:

```
gate = torch.sigmoid(x @ W_gate + state @ U_gate + b_gate)
candidate = torch.tanh(x @ W_in + state @ W_state + b_stat
e)
state = gate * state + (1.0 - gate) * candidate
output = state @ W_out
```

Process tokens sequentially in the reference implementation. Later optimize using chunked scans if profiling proves this layer is a bottleneck.

State rules:

- Initialize to zeros at document boundaries.
- Detach only when intentionally performing truncated backpropagation.
- Never leak state between unrelated samples.
- Save runtime conversation state separately from model checkpoints.

8.6 Feed-forward network

Use a gated SiLU projection:

```
hidden = silu(x @ W_gate) * (x @ W_up)
out = hidden @ W_down
```

8.7 Residual block

Use pre-normalization:

```
x = x + sequence_mixer(RMSNorm(x))
x = x + feed_forward(RMSNorm(x))
```

8.8 Retrieval cross-attention

The query sequence attends to encoded retrieved chunks. Apply a retrieval mask so padded memory positions cannot contribute. Include source-boundary embeddings and rank embeddings so the model can distinguish chunks.

8.9 Output heads

- **LM head:** projects hidden states to vocabulary logits; tie with token embeddings.

- **Retrieval head:** projects a pooled request representation to a normalized embedding.
- **Optional action classifier:** predicts the action family and can be used as an auxiliary loss.

8.10 Forward contract

```python
@dataclass
class ModelOutput:
    logits: torch.Tensor
    loss: torch.Tensor | None
    retrieval_embedding: torch.Tensor | None
    recurrent_state: list[torch.Tensor] | None
    auxiliary_losses: dict[str, torch.Tensor]
```

Input shapes:

```
input_ids:       [batch, sequence]
target_ids:      [batch, sequence] or None
attention_mask:  [batch, sequence]
memory_ids:      [batch, chunks, memory_sequence] or None
```

---

9. Training system

9.1 Objectives

Main language-model loss:

$$L_{LM} = CrossEntropy(logits, next\_token)$$

Retrieval contrastive loss:

$$L_{ret} = -log \frac{exp(sim(q, d^{+})/\tau)}{\sum_{j} exp(sim(q, d_j)/\tau)}$$

Optional action classification loss:

$$L = L_{LM} + \lambda_{ret} L_{ret} + \lambda_{action} L_{action}$$

Begin with only `L_LM`. Add one objective at a time and record an ablation.

9.2 Optimizer and schedule

- AdamW.
- Learning rate: `3e-4` initial full-model target.
- Betas: `(0.9, 0.95)`.
- Weight decay: `0.1`, excluding normalization scales and biases.
- Gradient clipping: global norm `1.0`.
- Linear warmup followed by cosine decay.
- Minimum learning rate: approximately 10% of peak.

9.3 Training step

```
Load batch
→ move batch to device
→ autocast forward pass
→ divide loss by accumulation steps
→ backward
→ on accumulation boundary:
    unscale gradients
    clip global norm
    optimizer step
    scheduler step
    clear gradients
→ record metrics
→ periodically evaluate and checkpoint
```

9.4 Memory controls

Apply in this order:

1. Mixed precision.
2. Reduce micro-batch size.
3. Increase gradient accumulation.
4. Activation checkpointing.
5. Reduce context length.
6. Reduce attention window.
7. Reduce layer count or width only as a final architectural change.

9.5 Checkpoint contents

Each atomic checkpoint directory contains:

```
model.safetensors
optimizer.pt
scheduler.pt
scaler.pt
trainer_state.json
config.resolved.yaml
tokenizer/
dataset_manifest.json
rng_state.pt
metrics.jsonl
```

Write to a temporary directory, validate all files, then rename atomically. Keep `latest` and `best-validation` pointers. Resume must restore Python, NumPy, CPU, and CUDA random states.

9.6 Logged metrics

- Train and validation loss.
- Perplexity with overflow-safe reporting.
- Tokens per second.
- Step duration.
- Learning rate.
- Gradient norm.
- Allocated and reserved device memory.
- Data-loading time.
- Action validity and retrieval metrics during evaluation.

---

10. External memory and retrieval

10.1 SQLite schema

```sql
CREATE TABLE documents (
    id INTEGER PRIMARY KEY,
    source_path TEXT NOT NULL,
```

```
    source_hash TEXT NOT NULL,
    language TEXT,
    indexed_at TEXT NOT NULL,
    UNIQUE(source_path, source_hash)
);

CREATE TABLE chunks (
    id INTEGER PRIMARY KEY,
    document_id INTEGER NOT NULL REFERENCES documents(id),
    chunk_index INTEGER NOT NULL,
    start_line INTEGER,
    end_line INTEGER,
    content TEXT NOT NULL,
    token_count INTEGER NOT NULL,
    content_hash TEXT NOT NULL UNIQUE
);

CREATE VIRTUAL TABLE chunks_fts USING fts5(
    content,
    content='chunks',
    content_rowid='id'
);

CREATE TABLE corrections (
    id INTEGER PRIMARY KEY,
    request TEXT NOT NULL,
    context TEXT,
    incorrect_output TEXT,
    corrected_output TEXT NOT NULL,
    created_at TEXT NOT NULL,
    training_status TEXT NOT NULL DEFAULT 'pending'
);
```

10.2 Chunking

Prefer semantic boundaries:

- Markdown: headings and paragraphs.
- Python: classes and functions.

- TypeScript: declarations, functions, and modules.
- Generic text: paragraphs with overlap.

Target 200–400 BPE tokens per chunk with 30–60 tokens of overlap. Preserve source path and line ranges.

10.3 Retrieval stages

Stage 1: SQLite FTS/BM25 or TF-IDF retrieval.

Stage 2: Learned query/chunk embeddings with contrastive training.

Stage 3: Hybrid lexical + dense score.

Stage 4: Lightweight reranker over the top candidates.

Hybrid score:

```
score = α × normalized_lexical_score
      + β × cosine_dense_score
      + γ × recency_or_path_prior
```

Tune weights on a held-out retrieval benchmark rather than intuition.

10.4 Grounded prompt format

```
<USER>
{request}
<MEMORY>
[1] {path}:{start_line}-{end_line}
{chunk}

[2] ...
</MEMORY>
<ASSISTANT>
```

The model must be trained to say when evidence is missing or contradictory.

---

11. Action system

11.1 Versioned schema

Use Pydantic discriminated unions. Initial action types:

- `search_text`

- `list_files`
- `read_file_section`
- `find_todos`
- `summarize_results`
- `ask_clarification`
- `refuse_action`

Example schema concept:

```
class SearchText(BaseModel):
    version: Literal["1"]
    action: Literal["search_text"]
    arguments: SearchTextArguments
```

Every schema must enforce path constraints, result limits, maximum pattern length, supported extensions, and strict unknown-field rejection.

11.2 Constrained decoding

Build a deterministic parser state machine that tracks:

- JSON object/array state.
- Allowed property names.
- Required properties.
- Enumerated action names.
- String escaping.
- Number and boolean syntax.
- Schema-specific value constraints where feasible.

At every generation step:

```
allowed = grammar.allowed_token_ids(parser_state)
masked_logits = torch.full_like(logits, float("-inf"))
masked_logits[allowed] = logits[allowed]
next_id = sample(masked_logits)
parser_state.consume(next_id)
```

Because BPE tokens can contain multiple bytes, validate transitions byte-by-byte before allowing a token.

11.3 Policy boundary

The executor, not the model, owns authorization.

Initial policy:

- Read-only actions only.
- Workspace-root path allowlist.
- Resolve symlinks before checking containment.
- Deny hidden secret files and configured patterns.
- Limit file size, output size, recursion depth, and execution time.
- No shell interpolation.
- No arbitrary regular expressions without timeout protection.
- Log action request, validated arguments, result summary, and rejection reason.

---

12. Runtime orchestration

```
1. Classify request as answer, retrieval, action, or clarification.
2. Retrieve relevant chunks when required.
3. Construct the versioned prompt.
4. Generate either text or a constrained action.
5. Validate action schema and policy.
6. Execute approved read-only operation.
7. Return bounded results to the model.
8. Generate grounded final response.
9. Record optional feedback and diagnostics.
```

Generation defaults:

```
max_new_tokens: 256
temperature_text: 0.8
top_k_text: 50
top_p_text: 0.95
temperature_action: 0.0
stop_tokens:
```

```
  - <EOS>
  - </ACTION>
```

Use greedy or deterministic decoding for actions. Use sampling only for ordinary text.

---

13. Evaluation plan

13.1 Language modeling

- Validation cross-entropy.
- Perplexity.
- Byte/token throughput.
- Repetition rate.
- Held-out code and documentation losses reported separately.

13.2 Action evaluation

- JSON validity.
- Schema validity.
- Exact action-type accuracy.
- Argument exact match and field-level F1.
- Policy-safe rate.
- Clarification accuracy for ambiguous requests.
- Denial accuracy for prohibited requests.

13.3 Retrieval evaluation

- Recall@1, Recall@5, Recall@10.
- Mean reciprocal rank.
- NDCG@10.
- Retrieval latency.
- Indexing throughput.

13.4 Grounded-answer evaluation

Create questions with known supporting chunks and measure:

- Answer correctness.
- Evidence recall.
- Unsupported-claim rate.
- Correct abstention when evidence is absent.
- Source-path and line-range accuracy.

13.5 Architecture ablations

Run comparable experiments:

1. Full causal attention baseline.
2. Local attention only.
3. Local attention + recurrent memory.
4. Hybrid model without retrieval cross-attention.
5. Hybrid model with retrieval cross-attention.
6. Byte tokenizer versus BPE.
7. Unconstrained versus constrained action generation.

Keep parameter count, token budget, and data order as close as possible.

13.6 Performance benchmark

Record:

- Peak training memory.
- Training tokens/second.
- First-token latency.
- Generation tokens/second.
- Retrieval latency by corpus size.
- Total runtime latency for answer and action flows.

---

14. Test strategy

14.1 Unit tests

- Tokenizer round-trip and serialization.
- Dataset shift alignment and boundary handling.
- Causal and local masks.
- RoPE cache behavior.
- RMSNorm numerical correctness.
- Attention shape and reference equivalence.
- Recurrent-state reset and isolation.
- Loss masking for padding.
- Grammar transition correctness.
- Path-policy containment and symlink handling.

14.2 Integration tests

- Raw document → cleaned corpus → token binary.
- Configuration → model → one optimizer step.
- Save checkpoint → reload → identical logits.
- Index project → retrieve known chunk.
- Request → constrained action → validator → executor → response.
- Correction → training example conversion.

14.3 Required invariants

- No future-token leakage.
- No recurrent state leakage between samples.
- Padding contributes neither attention nor loss.
- Validation and test documents never appear in training.
- Checkpoint resume reproduces the uninterrupted run within documented tolerance.
- Executor cannot escape its configured root.

14.4 Golden tiny-overfit test

Create a very small deterministic corpus and train until near-perfect memorization. Failure to overfit blocks all larger runs.

---

15. Implementation roadmap

Phase 0 — Foundation

- [ ] Initialize repository, dependency lock, linting, formatting, typing, and tests.
- [ ] Define typed configuration models and three execution profiles.
- [ ] Add deterministic seed handling and environment reporting.
- [ ] Add structured run directories and metric logging.

Exit: One command runs tests and another prints the resolved configuration.

Phase 1 — Byte-level bigram

- [ ] Implement byte tokenizer.
- [ ] Build memory-mapped dataset.
- [ ] Implement embedding + linear LM head.
- [ ] Write minimal trainer and generator.
- [ ] Overfit a tiny sample.

Exit: Loss falls sharply and checkpoint reload reproduces logits.

Phase 2 — Reference Transformer

- [ ] Implement RMSNorm, RoPE, causal attention, feed-forward, and residual blocks.
- [ ] Add causal-leakage tests.
- [ ] Add autoregressive generation with KV-cache interface.
- [ ] Train a 2–5M parameter model.

Exit: Stable training and coherent sample fragments.

Phase 3 — BPE and data pipeline

- [ ] Implement byte-backed BPE training and encoding.
- [ ] Add corpus manifests, cleaning, secret filtering, and deduplication.

- [ ] Build immutable train/validation/test binaries.
- [ ] Compare byte and BPE efficiency.

Exit: Reproducible tokenizer and dataset hashes.

Phase 4 — Optimized local attention

- [ ] Implement local attention masks.
- [ ] Add optimized attention backend.
- [ ] Compare outputs to reference implementation.
- [ ] Add mixed precision and activation checkpointing.

Exit: Small profile trains without numerical instability.

Phase 5 — Recurrent memory

- [ ] Implement reference gated recurrent layer.
- [ ] Add state isolation and reset tests.
- [ ] Build alternating hybrid blocks.
- [ ] Benchmark against equal-parameter local-attention baseline.

Exit: Hybrid model demonstrates either better long-context quality or lower memory at comparable quality.

Phase 6 — Action generation

- [ ] Define versioned Pydantic schemas.
- [ ] Generate and validate synthetic action data.
- [ ] Implement grammar parser and token filtering.
- [ ] Add policy layer and read-only executor.
- [ ] Build action benchmark.

Exit: 98%+ valid structured output and zero policy bypasses in tests.

Phase 7 — External retrieval

- [ ] Implement chunkers and SQLite store.
- [ ] Add FTS/BM25 or TF-IDF retrieval.

- [ ] Create retrieval benchmark.
- [ ] Add grounded prompt formatting.
- [ ] Train abstention examples.

Exit: Retrieval-augmented answers beat closed-book answers.

Phase 8 — Learned retrieval

- [ ] Add retrieval projection head.
- [ ] Construct positive and hard-negative pairs.
- [ ] Train contrastive objective.
- [ ] Build hybrid lexical+dense ranking.
- [ ] Add retrieval cross-attention.

Exit: Recall and grounded-answer quality improve on held-out cases.

Phase 9 — Full training run

- [ ] Freeze code and dataset versions.
- [ ] Run golden overfit and resume tests.
- [ ] Train full profile with periodic evaluation.
- [ ] Preserve best-validation and final checkpoints.
- [ ] Produce an experiment report with ablations.

Exit: All release gates pass.

Phase 10 — Local product interface

- [ ] Build CLI chat and indexing commands.
- [ ] Display retrieved sources and action previews.
- [ ] Add feedback/correction capture.
- [ ] Add bounded session state.
- [ ] Package reproducible installation instructions.

Exit: A clean installation can index a project and complete the benchmark workflow end-to-end.

---

16. Command interface

Recommended commands:

```
# Environment and tests
make setup
make test

# Data and tokenizer
python scripts/prepare_corpus.py --config configs/small.yam
l
python scripts/train_tokenizer.py --config configs/small.ya
ml
python scripts/tokenize_corpus.py --config configs/small.ya
ml

# Training and evaluation
python scripts/train.py --config configs/debug.yaml
python scripts/train.py --config configs/full.yaml --resume
checkpoints/latest
python scripts/evaluate.py --checkpoint checkpoints/best-va
lidation

# Retrieval and use
python scripts/index_workspace.py --root ./example-project
python scripts/chat.py --checkpoint checkpoints/best-valida
tion
```

Every command must support `--help`, validate configuration before work begins, and exit non-zero on failure.

---

17. Debugging guide

Loss does not decrease

Check, in order:

1. Input-target shift.
2. Padding loss mask.

3. Causal mask orientation.
4. Learning rate and gradient norm.
5. Whether parameters receive nonzero gradients.
6. Whether one tiny batch can be overfit.
7. Tokenizer/data corruption.

Loss becomes NaN

- Run the same batch in float32.
- Lower learning rate.
- Confirm attention rows are not completely masked.
- Apply stable softmax and RMSNorm epsilon.
- Inspect logits and gradients for the first non-finite tensor.
- Verify gradient scaler handling.

Device memory exhaustion

- Reduce micro-batch size.
- Enable activation checkpointing.
- Reduce context length or attention window.
- Ensure evaluation disables gradients.
- Delete retained tensors in metric logging.
- Check for growing KV or recurrent-state caches.

Model generates repetitive text

- Verify training data is deduplicated.
- Compare greedy and sampled decoding.
- Adjust temperature, top-k, and top-p.
- Inspect whether the model is undertrained or overfitting.
- Add repetition metrics; do not hide the issue only with a repetition penalty.

Retrieval hurts answers

- Inspect retrieved chunks manually.
- Evaluate retrieval separately from generation.
- Reduce chunk size or improve boundaries.
- Add hard negatives.
- Train examples with irrelevant retrieved context.
- Add an explicit no-useful-evidence behavior.

---

18. Security, privacy, and data governance

- Treat indexed files and generated actions as untrusted input.
- Keep the executor separate from the model process where practical.
- Use allowlists rather than denylists for executable capabilities.
- Never pass generated text to a shell.
- Redact secrets before logs, prompts, datasets, and feedback storage.
- Record dataset provenance and licenses.
- Make deletion possible by source identifier and content hash.
- Bind indexes to an explicit workspace root.
- Include adversarial tests for traversal, symlinks, huge files, recursive trees, malformed JSON, prompt injection inside documents, and output flooding.
- Clearly distinguish retrieved document instructions from trusted runtime policy.

---

19. Release gates

Model gate

- [ ] Tiny-overfit test passes.
- [ ] Causal leakage test passes.
- [ ] Validation loss improves over bigram baseline.
- [ ] Checkpoint resume test passes.
- [ ] No non-finite values in final evaluation.

Retrieval gate

- [ ] Recall@5 meets the chosen benchmark target.
- [ ] Missing-evidence abstention is evaluated.
- [ ] Source metadata is preserved through answer generation.

Action gate

- [ ] 98%+ syntactic validity.
- [ ] 100% schema validation before execution.
- [ ] Path traversal and symlink tests pass.
- [ ] No state-changing action is available.
- [ ] All execution outputs are bounded.

Reproducibility gate

- [ ] Locked dependencies.
- [ ] Dataset and tokenizer hashes recorded.
- [ ] Resolved configuration saved.
- [ ] Fresh installation instructions tested.
- [ ] Benchmark report generated from a clean checkout.

---

20. Final definition of done

PocketMind v1 is complete when a clean installation can:

1. Prepare a licensed, filtered corpus.
2. Train and serialize a byte-backed BPE tokenizer.
3. Train the hybrid model from an initialization seed.
4. Stop and resume training from an atomic checkpoint.
5. Evaluate language, retrieval, grounding, action, and performance metrics.
6. Index a local project while excluding secrets and unsupported files.
7. Retrieve relevant passages for natural-language questions.
8. Produce grounded answers with source locations.

9. Generate validated, grammar-constrained read-only actions.
10. Execute those actions inside a strict workspace boundary.
11. Capture corrections as versioned future training examples.
12. Reproduce the released benchmark results from recorded artifacts.

> 🏁 **Build order rule:** correctness → tests → observability → baseline → optimization → architectural experimentation → product integration. Do not begin a large training run until the tiny-overfit, causal-mask, checkpoint-resume, and dataset-split tests all pass.

📚 PocketMind Prerequisites — AI & LLM Terms and Techniques